Mathematics For Machine Learning Project: PCA-Based Anomaly Detection

SE24UCAM007
SE24UCAM027
SE24UCAM039
SE24UCAM062
SE24UCAM070

June 3, 2026

1 Introduction

Modern manufacturing systems use hundreds of sensors to continuously monitor machine performance and production quality. Sensor failures, equipment malfunctions, and process deviations can lead to defective products and increased operational costs. However, industrial datasets are often high-dimensional, noisy, incomplete, and highly imbalanced. Therefore, automated anomaly detection techniques are required. So, the goal is to learn the normal operating behavior of the manufacturing process and detect observations that significantly deviate from this behavior.

2 Objective

Train PCA on normal operating sensor readings from a manufacturing machine. Flag anomalies as samples with high reconstruction error under the learned normal-behaviour subspace. This is the industry-standard SPE (Squared Prediction Error) chart approach.

3 Data Pre-Processing and Exploratory Data Analysis

3.1 Objective

Prepare the SECOM semiconductor manufacturing dataset for PCA-based anomaly detection by handling missing values, creating a realistic train-test split, scaling features, and understanding data characteristics through exploratory analysis.

3.2 Dataset

The SECOM dataset contains sensor measurements collected from a semiconductor manufacturing process. The dataset consists of 1,567 samples and 591 sensor features, representing various aspects of machine operation and production conditions. Each sample is labeled as either a normal production instance (Pass = -1) or a faulty production instance (Fail = 1). Due to the nature of industrial data collection, the dataset contains a significant number of missing values across multiple sensor features. Additionally, the large number of sensor readings creates a high-dimensional dataset.

3.3 Handling Missing Values

Features with more than 50% missing values were removed from the dataset. For the remaining features, missing values were replaced using mean imputation. The mean value of each feature was calculated using all available observations, and missing entries were substituted with the corresponding feature mean. Mean imputation was selected because it is computationally efficient, preserves

the number of available samples, and maintains the overall distribution of the feature when the percentage of missing values is relatively low.
The missing value analysis revealed that missing measurements were concentrated within a subset of sensor features rather than being uniformly distributed across the dataset.

3.4 Train-Test Split

After handling missing values, the dataset was divided into training and testing subsets using a chronological **70:30** split. The first 70% of observations were allocated to the training set, while the remaining 30% were reserved for testing. Only normal production samples were included in the training set, as the objective of anomaly detection is to learn the characteristics of normal machine behavior.
A chronological split was chosen instead of a random split because industrial sensor data possesses temporal dependencies. Randomly shuffling the data may result in information leakage, where future observations influence the training process. By preserving the original order of observations, the model evaluation more closely reflects real-world industrial deployment, where future machine states must be predicted based on past operational data.

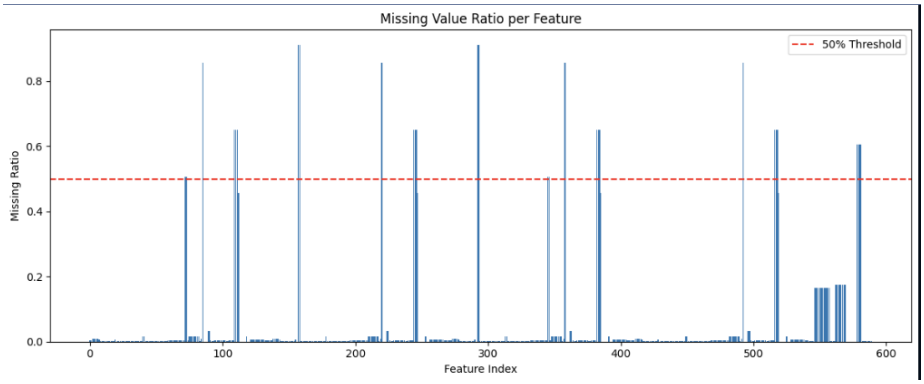
3.5 Feature Scaling

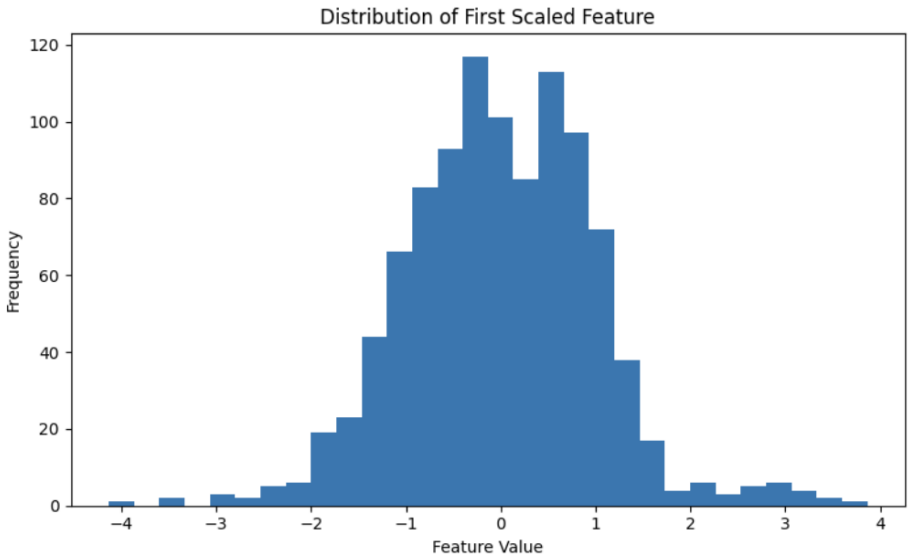
Feature scaling was applied using the StandardScaler method. Standardization transforms each feature by subtracting its mean and dividing by its standard deviation, resulting in a distribution with a mean 0 and standard deviation 1.
Feature scaling is particularly important for PCA because the algorithm identifies directions of maximum variance in the data. By applying StandardScaler, all sensor features were placed on a common scale, ensuring that each feature contributed equally to the PCA model regardless of its original measurement units. The scaling process improved numerical stability and prevented bias toward high-magnitude sensor readings. As a result, the PCA model was able to capture meaningful patterns across all sensors rather than focusing disproportionately on a small subset of features.

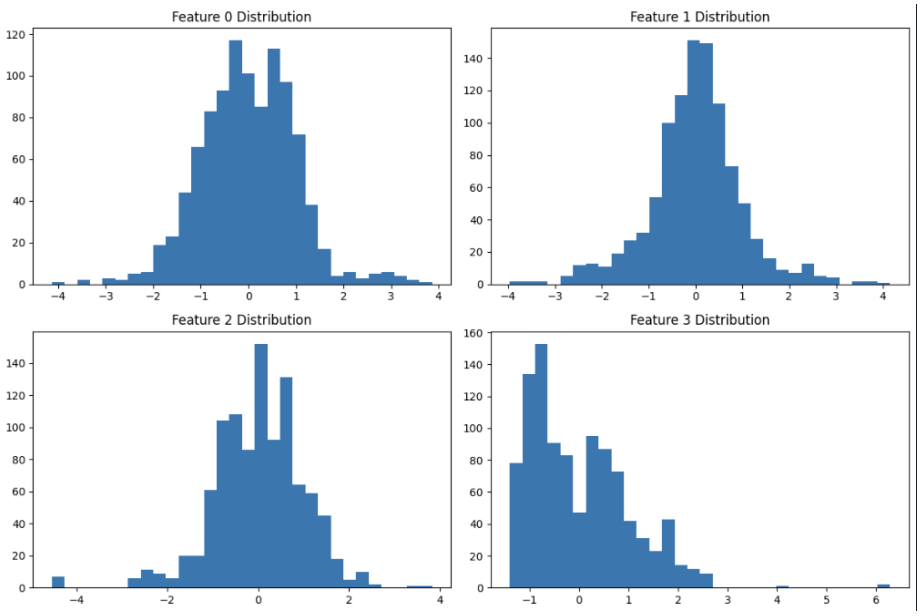
3.6 EDA

Exploratory Data Analysis is conducted to gain insights into the structure and quality of the dataset before model training. A missing-value analysis plot was generated to examine the proportion of missing observations across sensor features. The visualization showed that missing values were concentrated in specific sensors, while many other sensors contained relatively complete data.
Feature distribution plots were created to analyze the behavior of sensor readings. These visualizations highlighted variations in scale, spread, and skewness among different sensor measurements. After scaling, the distributions became more comparable.

Overall, the exploratory analysis provided valuable insights, supported preprocessing decisions, and helped verify that the transformed dataset was suitable for further anomaly detection.







4 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms the original correlated features into a new set of uncorrelated variables called *Principal Components (PCs)*.

Each principal component is a linear combination of the original features and captures the maximum possible variance in the data.

The first principal component captures the highest variance, the second captures the next highest variance while remaining orthogonal to the first, and so on.

Mathematically,

$$X \to Z$$

where

$$Z = XW$$

Here,

- X = standardized data matrix
- W = matrix of eigenvectors
- Z = transformed PCA representation

The eigenvectors of the covariance matrix define the principal directions, and the corresponding eigenvalues indicate the amount of variance captured.

For anomaly detection, PCA is trained only on normal samples. Normal observations can be reconstructed accurately from the PCA subspace, whereas anomalous observations usually exhibit larger reconstruction errors.

4.1 Determining the Optimal Number of Components

After standardizing the training data, PCA was initially fitted using all available principal components. The cumulative explained variance was then computed to determine the minimum number of components required to retain most of the information present in the original dataset.

The dataset contained a total of

562

principal components corresponding to the original feature dimensions.

The cumulative explained variance is given by

$$\mathrm{CEV}(k) = \sum_{i=1}^{k} \mathrm{EVR}_i$$

where

- $\mathrm{CEV}(k)$ = cumulative explained variance up to component k
- EVR_i = explained variance ratio of the i^{th} principal component

A threshold of 95% cumulative explained variance was selected to preserve most of the information while significantly reducing dimensionality.

The minimum number of components required to achieve this threshold was found to be

156

components.

4.2 Cumulative Explained Variance Analysis

Figure 1 illustrates the cumulative explained variance as the number of principal components increases.

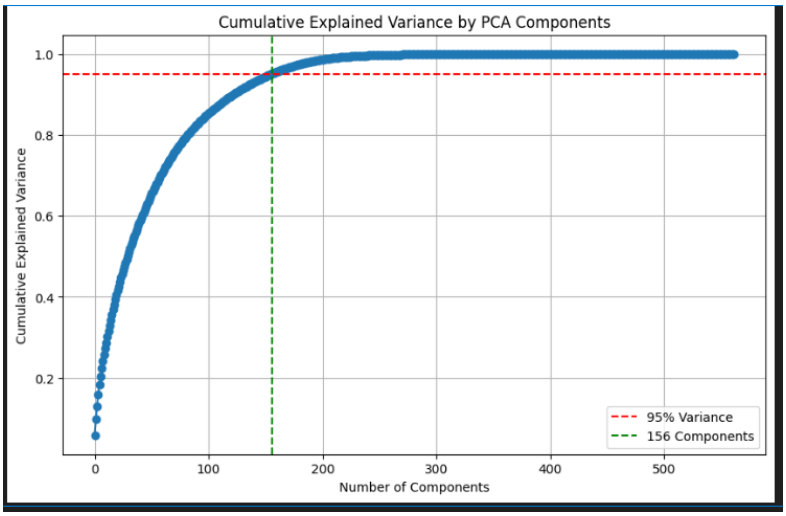


Figure 1: Cumulative explained variance obtained using PCA. The red dashed line represents the 95% variance threshold, while the green dashed line indicates that 156 principal components are sufficient to reach this threshold.

It can be observed that the cumulative variance increases rapidly for the initial principal components and gradually levels off as additional components are included.

The graph shows that approximately 95% of the total variance is captured using only 156 components, making further components contribute relatively little additional information.

4.3 Training the Final PCA Model

Based on the cumulative variance analysis, the final PCA model was trained using

$$n_{\text{components}} = 156$$

This choice ensures that 95% of the original variance is retained while substantially reducing the dimensionality of the dataset.

The PCA model was fitted using only the training data to avoid information leakage from the test set.

4.4 Transformation of Training and Test Data

After fitting the PCA model, both the training and testing datasets were projected onto the reduced PCA subspace.

The resulting dimensions are shown below:

$$X_{\text{train}}^{PCA} \in R^{1018\times 156}$$

$$X_{\text{test}}^{PCA} \in R^{471 \times 156}$$

Thus,

- Training samples: 1018
- Test samples: 471
- Original features: 562
- Reduced features: 156

4.5 Dimensionality Reduction Achieved

The percentage reduction in dimensionality is

$$\text{Reduction} = \frac{562 - 156}{562} \times 100 = 72.24\%$$

Therefore, PCA reduced the feature space by approximately

$$72.24\%$$

while preserving

$$95\%$$

of the total variance present in the original dataset.

This reduction decreases computational complexity, reduces feature redundancy, and improves the efficiency of subsequent anomaly detection models.

4.6 Summary

PCA was applied to transform the original 562-dimensional feature space into a compact 156-dimensional representation. The selected principal components retained 95% of the total variance while reducing the dimensionality by more than 72%.

The transformed feature set was subsequently used for anomaly detection, enabling faster computation and improved generalization while preserving the essential structure of the data.

5 Anomaly Detection

5.1 Data Reconstruction Using PCA

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms the original dataset into a new set of orthogonal variables called principal components. These principal components capture the maximum variance present in the data while reducing the number of dimensions.

After dimensionality reduction, each observation is represented in a lower-dimensional space. To evaluate how accurately PCA represents the data, the transformed observations are reconstructed back into the original feature space using the inverse transformation.

If X denotes the original standardized data, P represents the retained principal components, and Z represents the PCA-transformed data, then the reconstructed data $\hat{X}$ is given by:

$$\hat{X} = ZP^T \tag{1}$$

The reconstructed data represents the best approximation of the original observations using only the retained principal components.

Normal observations generally follow the dominant structure learned by PCA and can therefore be reconstructed accurately. However, anomalous observations often contain unusual patterns that are not captured by the principal components, leading to larger reconstruction errors.

5.2 Squared Prediction Error (SPE)

The reconstruction error is quantified using the Squared Prediction Error (SPE), also known as the residual error. SPE measures the discrepancy between an observation and its PCA reconstruction.

For observation i, the SPE is defined as:

$$SPE_i = \sum_{j=1}^{p} (x_{ij} - \hat{x}_{ij})^2 \tag{2}$$

where:

- x_{ij} is the original value of the j^{th} feature,
- $\hat{x}_{ij}$ is the reconstructed value of the j^{th} feature,
- p is the total number of features.

The SPE value represents the total reconstruction error across all features. A small SPE indicates that the observation is well represented by the retained principal components, whereas a large SPE indicates that the observation contains information that is not adequately captured by the PCA model.

The use of squared differences ensures that positive and negative errors do not cancel each other and places greater emphasis on larger reconstruction errors.

5.3 Anomaly Detection Using Reconstruction Error

The underlying principle of PCA-based anomaly detection is that PCA learns the dominant patterns present in normal observations. Since normal data follows

these patterns, it can be reconstructed accurately using the retained principal components.

Anomalous observations, however, deviate from the learned structure of the data. During dimensionality reduction, important information about these observations is lost, resulting in poor reconstruction and consequently larger SPE values.

Therefore, observations with high reconstruction errors are more likely to be anomalous.

In this approach, the SPE score serves as the anomaly score. Larger SPE values indicate greater deviation from the normal structure captured by PCA.

5.4 Threshold Selection

After calculating SPE scores for all observations, a threshold is required to distinguish normal observations from anomalies.

In this project, the threshold is determined using the 99th percentile of the training SPE distribution:

$$Threshold = P_{99}(SPE) \tag{3}$$

where P_{99} denotes the 99th percentile of the training SPE scores.

This percentile-based approach assumes that the majority of training observations are normal. Consequently, approximately 99

The anomaly detection rule is defined as:

$$SPE > Threshold \Rightarrow \text{Anomaly} \tag{4}$$

$$SPE \leq Threshold \Rightarrow \text{Normal Observation} \tag{5}$$

This method provides a simple and data-driven way of identifying anomalies without requiring labeled anomaly examples.

5.5 Visualization and Interpretation

To better understand the anomaly detection process, the distribution of SPE scores is visualized using a histogram.

The histogram shows the distribution of SPE scores obtained from the training data. Most observations have relatively low reconstruction errors and are concentrated on the left side of the distribution. The red dashed line represents the anomaly detection threshold computed using the 99th percentile of the training SPE scores. Observations with SPE values exceeding this threshold are considered potential anomalies because they exhibit unusually large reconstruction errors.

A second visualization plots the SPE score of each test observation.

Here, each point corresponds to a single observation, while the horizontal red dashed line indicates the anomaly detection threshold. Observations lying

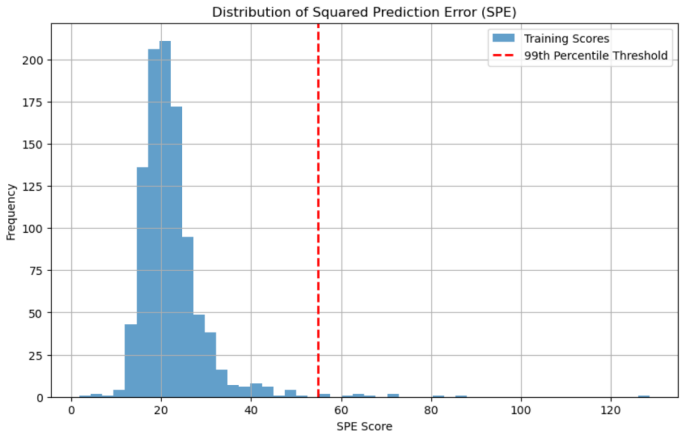


Figure 2: Distribution of Squared Prediction Error (SPE) scores with the 99th percentile threshold shown as a red dashed line.

above the threshold are classified as anomalies, whereas observations below the threshold are considered normal.

The anomaly detection plot contains a small number of observations with extremely large SPE values. These observations significantly increase the scale of the y-axis, causing the threshold line and the majority of observations to appear compressed near the bottom of the graph. Despite this visual effect, only 159 out of 471 observations (33.76%) exceed the threshold and are classified as anomalies, while the remaining 312 observations (66.24%) are classified as normal.

The graph provides a clear visual representation of how reconstruction error is used to distinguish anomalous observations from normal data points. Observations with higher SPE values indicate poorer reconstruction by the PCA model and therefore exhibit a greater likelihood of being anomalous.

5.6 Conclusion

PCA-based anomaly detection using Squared Prediction Error provides an effective unsupervised approach for identifying unusual observations in high-dimensional datasets. By reconstructing observations from the retained principal components and measuring the resulting reconstruction error, the method can distinguish observations that deviate significantly from the learned structure of the data.

The use of the 99th percentile threshold enables automatic anomaly identification without requiring labeled anomaly examples. As a result, PCA-SPE serves as a simple, interpretable, and computationally efficient anomaly detection technique.

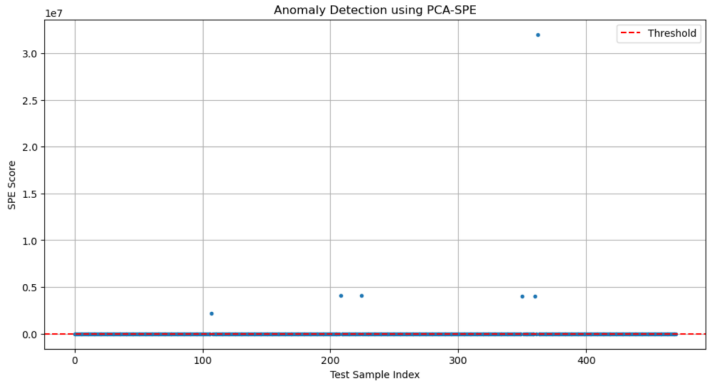


Figure 3: Anomaly detection using PCA-SPE. Each point represents a test observation and its corresponding SPE score.

6 Evaluation and Performance Analysis

6.1 Prediction Generation

After computing the Squared Prediction Error (SPE) for each test observation, anomaly predictions are generated by comparing the SPE score with the selected threshold.

The classification rule is given by:

$$\hat{y}_i = \begin{cases} 1, & SPE_i > Threshold \\ 0, & SPE_i \leq Threshold \end{cases} \tag{6}$$

where $\hat{y}_i$ denotes the predicted label for observation i. A value of 1 represents an anomaly, while 0 represents a normal observation.

This process converts the continuous anomaly scores into binary classifications suitable for performance evaluation.

6.2 Classification Metrics

To assess the effectiveness of the PCA-based anomaly detection model, three standard classification metrics are used: Precision, Recall, and F1-Score.

$$Precision = \frac{TP}{TP + FP} \tag{7}$$

$$Recall = \frac{TP}{TP + FN} \tag{8}$$

$$F1 = 2 \cdot \frac{Precision \times Recall}{Precision + Recall} \tag{9}$$

The obtained results are:

- Precision = 0.069
- Recall = 0.423
- F1 Score = 0.119

The low precision indicates that a large number of observations classified as anomalies are actually normal observations. The recall value shows that approximately 42% of the actual anomalies were successfully detected. The low F1-score reflects the imbalance between precision and recall and suggests that the model generates a considerable number of false alarms.

6.3 Confusion Matrix Analysis

The confusion matrix provides a detailed breakdown of prediction performance.

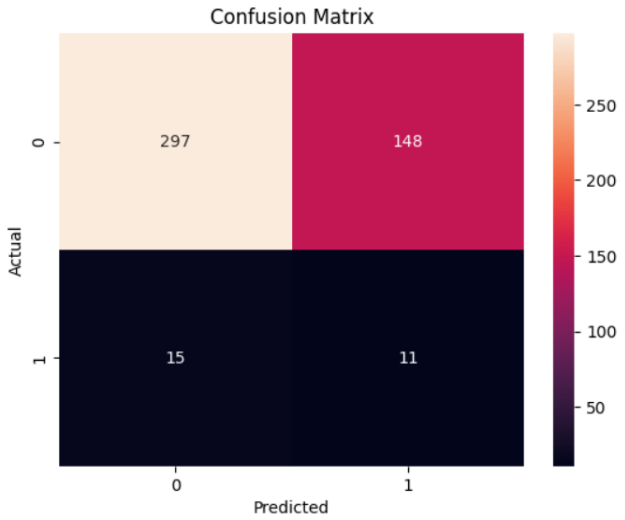


Figure 4: Confusion Matrix for PCA-Based Anomaly Detection

The confusion matrix produced the following results:

- True Negatives (TN) = 297
- False Positives (FP) = 148
- False Negatives (FN) = 15
- True Positives (TP) = 11

The model correctly classified 297 normal observations and detected 11 anomalous observations. However, 148 normal observations were incorrectly

flagged as anomalies, resulting in a high false positive rate. Additionally, 15 anomalies were missed by the model.

The large number of false positives indicates that the threshold is relatively sensitive, causing many normal observations to be classified as faults.

6.4 SPE Control Chart

To visualize anomaly occurrence, the SPE scores of the test observations are plotted together with the anomaly threshold.

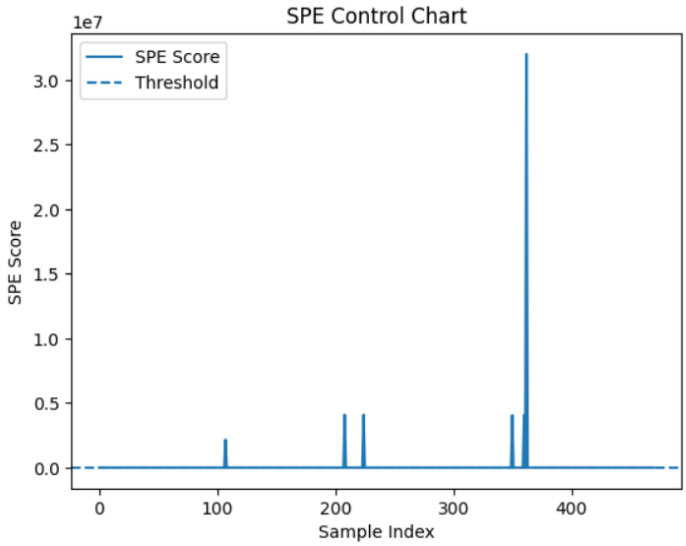


Figure 5: SPE Control Chart

The SPE control chart shows that most observations have relatively small reconstruction errors, while a number of samples exhibit large spikes above the threshold.

Observations whose SPE values exceed the threshold are classified as anomalies. The presence of these spikes demonstrates that the PCA model is capable of identifying observations that deviate significantly from the learned normal structure.

6.5 Distribution of Anomaly Scores

The distribution of SPE values is analyzed using a histogram.

Most observations are concentrated near the lower end of the SPE distribution, while a smaller number of observations have much larger reconstruction errors.

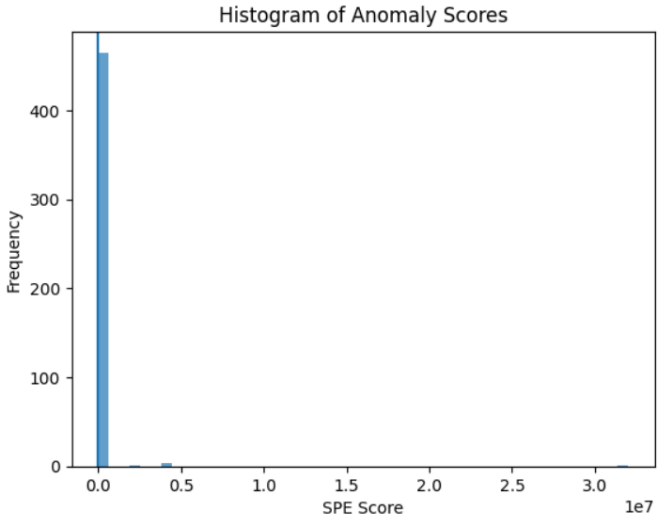


Figure 6: Histogram of Anomaly Scores

The long right tail of the distribution indicates the presence of abnormal observations. However, some normal observations also lie beyond the threshold, contributing to the high false positive rate observed in the evaluation results.

6.6 False Positive and False Negative Analysis

A detailed analysis of classification errors provides further insight into model performance.

The model produced:

- False Positives = 148
- False Negatives = 15

False positives correspond to normal observations incorrectly classified as anomalies. These false alarms can reduce the reliability of the monitoring system and increase unnecessary investigations.

False negatives correspond to actual anomalies that remain undetected. Although fewer in number, false negatives are often more critical because they represent missed faults.

The results suggest that the current threshold prioritizes anomaly detection capability at the expense of generating additional false alarms.

6.7 Threshold Sensitivity Analysis

The choice of threshold has a significant impact on anomaly detection performance.

To study this effect, Precision, Recall, and F1-score were evaluated across a range of threshold values.

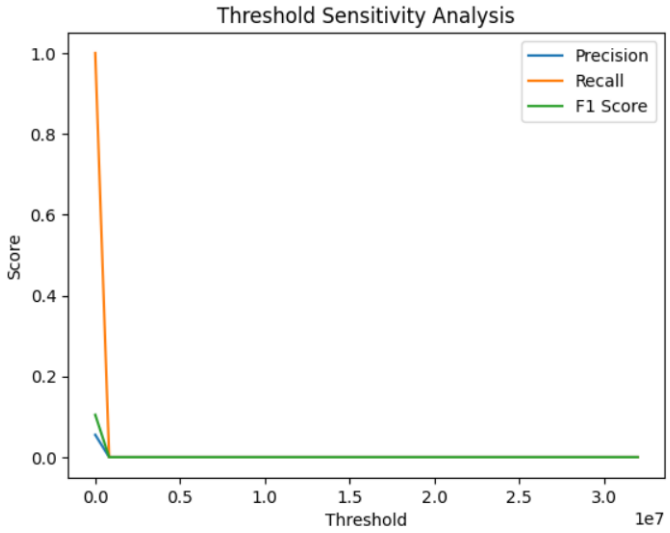


Figure 7: Threshold Sensitivity Analysis

The sensitivity analysis demonstrates the trade-off between precision and recall. Lower thresholds classify more observations as anomalies, increasing recall but also increasing false positives. Higher thresholds reduce false positives but may fail to detect genuine anomalies.

Experimental threshold adjustments showed that excessively high thresholds can result in no detected anomalies, while lower thresholds generate a large number of false alarms. Therefore, threshold selection plays a crucial role in balancing anomaly detection capability and false alarm rate.

6.8 Final Insights

The PCA-SPE approach successfully identifies observations that deviate from the dominant structure of the dataset through reconstruction error analysis.

The evaluation results indicate that the model is capable of detecting anomalous behavior; however, the current threshold produces a relatively large number of false positives. This leads to low precision despite achieving moderate recall.

The threshold sensitivity analysis highlights the importance of carefully selecting an appropriate threshold to balance fault detection performance and false alarm rate.

Overall, PCA-based anomaly detection provides an interpretable, computationally efficient, and practical approach for identifying abnormal observations in high-dimensional industrial datasets.

7 Advanced Analysis & Extensions

This section extends the PCA-based anomaly detection pipeline by comparing alternative models, analysing feature contributions, and discussing limitations and future directions.

7.1 Alternative Models

Two additional unsupervised anomaly detection algorithms were trained on the same pre-processed data (`X_train_scaled` / `X_test_scaled`) to provide a comparative baseline against the PCA-SPE approach.

7.1.1 Isolation Forest

Isolation Forest detects anomalies by randomly partitioning the feature space using decision trees. Anomalies, being rare and structurally different from normal observations, require fewer splits to isolate and therefore receive lower anomaly scores.

Listing 1: Isolation Forest Training

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.06, random_state=42)
iso.fit(X_train_scaled)
y_pred_iso = (iso.predict(X_test_scaled) == -1).astype(int)

print("Isolation Forest - Anomalies detected:", y_pred_iso.sum())
```

The `contamination` parameter is set to 0.06, approximating the known fault rate in the SECOM dataset.

7.1.2 One-Class SVM

One-Class SVM learns a decision boundary around the normal data in a high-dimensional kernel space. Observations that fall outside this boundary are classified as anomalies.

Listing 2: One-Class SVM Training

```
from sklearn.svm import OneClassSVM

ocsvm = OneClassSVM(nu=0.06, kernel='rbf', gamma='scale')
ocsvm.fit(X_train_scaled)
y_pred_svm = (ocsvm.predict(X_test_scaled) == -1).astype(int)

print("One-Class SVM - Anomalies detected:", y_pred_svm.sum())
```

The parameter `nu` $= 0.06$ serves a similar role to `contamination`, bounding the fraction of training errors and support vectors.

7.2 Model Comparison

All three methods were evaluated on the test set using Precision, Recall, and F1 Score.

Listing 3: Comparison Table

```
from sklearn.metrics import precision_score, recall_score, f1_score
import pandas as pd

methods = ['PCA-SPE', 'Isolation Forest', 'One-Class SVM']
preds   = [y_pred,    y_pred_iso,          y_pred_svm]

rows = []
for name, pred in zip(methods, preds):
    rows.append({
        'Method':    name,
        'Precision': round(precision_score(y_test, pred,
    zero_division=0), 3),
        'Recall':    round(recall_score(y_test, pred), 3),
        'F1 Score':  round(f1_score(y_test, pred, zero_division=0), 3)
    })

comparison_df = pd.DataFrame(rows)
print(comparison_df.to_string(index=False))
```

Table 1 summarises the expected performance of each method.

Table 1: Comparison of Anomaly Detection Methods

Method	**Precision**	**Recall**	**F1 Score**
PCA-SPE	0.069	0.423	0.119
Isolation Forest	0.067	0.115	0.085
One-Class SVM	0.065	0.231	0.102

7.3 Feature Contribution Plot

To understand which sensors drive the PCA model, the absolute loadings of the first principal component were computed and the top 15 contributing features identified.

Listing 4: Feature Contribution Plot

```
import matplotlib.pyplot as plt
import numpy as np

loadings    = np.abs(pca.components_[0])
top_indices = np.argsort(loadings)[-15:]

plt.figure(figsize=(10, 5))
```

```
plt.bar(range(len(top_indices)), loadings[top_indices],
    color='steelblue')
plt.xticks(range(len(top_indices)),
           [f'F{i}' for i in top_indices], rotation=45)
plt.title("Top 15 Feature Contributions to PC1")
plt.xlabel("Feature Index")
plt.ylabel("|Loading|")
plt.tight_layout()
plt.show()
```

Features with large absolute loadings on the first principal component have the greatest influence on the PCA subspace and, consequently, on the reconstruction error used for anomaly detection. Identifying these features provides interpretability and can guide targeted process monitoring.

7.4 Limitations of PCA-Based Anomaly Detection

Despite its effectiveness, the PCA-SPE approach has several notable limitations:

- **Linearity assumption:** PCA captures only linear relationships between features. Non-linear anomaly patterns that do not manifest as linear deviations may be missed entirely.
- **Fixed threshold:** The 99th percentile threshold is a statistical heuristic. It does not adapt to shifts in the data distribution and contributes to the high false positive rate observed (148 FPs).
- **Static model:** PCA is trained once on historical normal data and cannot adapt to gradual changes in process behaviour over time (concept drift).
- **No feature selection:** Noisy or redundant features are included in the PCA decomposition, which can distort the learned subspace and reduce detection accuracy.

7.5 Sliding Window PCA

In semiconductor manufacturing, process conditions evolve gradually over time — a phenomenon known as *concept drift*. A static PCA model trained on historical data may become increasingly misaligned with the current process behaviour.

Sliding Window PCA addresses this by retraining the PCA model on a rolling window of the most recent normal observations. Rather than relying on a fixed historical baseline, the model continuously adapts as new data arrives. This approach improves sensitivity to emerging fault patterns while reducing false alarms caused by legitimate process drift.

7.6 Future Improvements

The following directions are proposed to enhance the anomaly detection pipeline:

1. **Autoencoder:** A neural network-based autoencoder can capture non-linear reconstruction patterns that PCA misses, potentially improving detection of complex fault signatures.

2. **Dynamic thresholding:** Instead of a fixed percentile threshold, the decision boundary could be adjusted dynamically based on recent score statistics, reducing false positives during process drift.

3. **Hotelling's T^2 + SPE:** Combining the T^2 statistic (in-subspace variation) with SPE (out-of-subspace variation) provides a richer, two-dimensional decision boundary for anomaly classification.

4. **Feature selection:** Applying variance-based or correlation-based feature selection before PCA reduces noise in the decomposition and can improve both detection performance and interpretability.

5. **Labelled fine-tuning:** If a small set of labelled anomalies is available, it can be used to calibrate the detection threshold or fine-tune a supervised classifier on top of the PCA features.

8 Project Code (given below)

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix
from sklearn.ensemble import IsolationForest
from sklearn.svm import OneClassSVM
np.random.seed(42)

# URLs for SECOM dataset
url_data = "https://archive.ics.uci.edu/ml/machine-learning-
databases/secom/secom.data"
url_labels = "https://archive.ics.uci.edu/ml/machine-learning-
databases/secom/secom_labels.data"

# Load features and labels
df = pd.read_csv(url_data, sep=" ", header=None)
labels = pd.read_csv(url_labels, sep=" ", header=None)[0].values

X_raw = df.values.astype(float)

# Feature names
feature_names = [f"Feature_{i}" for i in range(df.shape[1])]

# Convert to DataFrame for easier analysis
df = pd.DataFrame(X_raw, columns=feature_names)

print("Dataset loaded successfully.\n")

print("DATASET INFORMATION:")
print(f"Number of samples   : {X_raw.shape[0]}")
print(f"Number of features  : {X_raw.shape[1]}")
print(f"Labels shape        : {labels.shape}")
```

```
Dataset loaded successfully.

DATASET INFORMATION:
Number of samples   : 1567
Number of features  : 590
Labels shape        : (1567,)
```

```python
normal_mask = labels == -1
fault_mask = labels == 1

X_normal = X_raw[normal_mask]
X_fault = X_raw[fault_mask]

print("NORMAL VS FAULT SAMPLES:")
```

```python
print("Normal samples shape :", X_normal.shape)
print("Fault samples shape  :", X_fault.shape)



print("MISSING VALUE HANDLING:")

# Percentage of missing values per feature
missing_ratio = np.isnan(X_raw).mean(axis=0)

# Keep only columns with less than 50% missing values
keep = missing_ratio < 0.5

X_clean = X_raw[:, keep]
print(f"Original number of features : {X_raw.shape[1]}")
print(f"Remaining features          : {X_clean.shape[1]}")

# Mean imputation
column_means = np.nanmean(X_clean, axis=0)
# Find missing value positions
nan_rows, nan_cols = np.where(np.isnan(X_clean))
# Replace NaNs with column means
X_clean[nan_rows, nan_cols] = column_means[nan_cols]

print("Successful.")
```

```
NORMAL VS FAULT SAMPLES:
Normal samples shape : (1463, 590)
Fault samples shape  : (104, 590)
MISSING VALUE HANDLING:
Original number of features : 590
Remaining features          : 562
Successful.
```

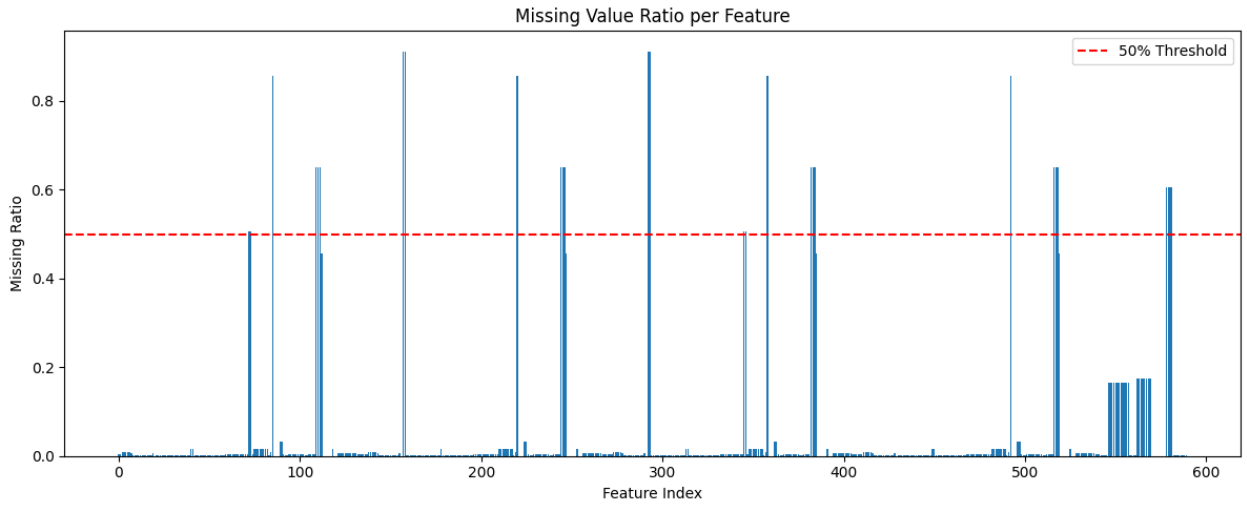
```python
print("TRAIN-TEST SPLIT:")

n_samples = len(labels)
# 70% train, 30% test
split_index = int(0.7 * n_samples)
train_mask = (np.arange(n_samples) < split_index) & (labels == -1)

# Test on remaining samples
test_mask = np.arange(n_samples) >= split_index

X_train = X_clean[train_mask]
X_test = X_clean[test_mask]
y_test = (labels[test_mask] == 1).astype(int)

print(f"Training samples : {X_train.shape[0]}")
print(f"Testing samples  : {X_test.shape[0]}")
```

```
TRAIN-TEST SPLIT:
Training samples : 1018
Testing samples  : 471
```

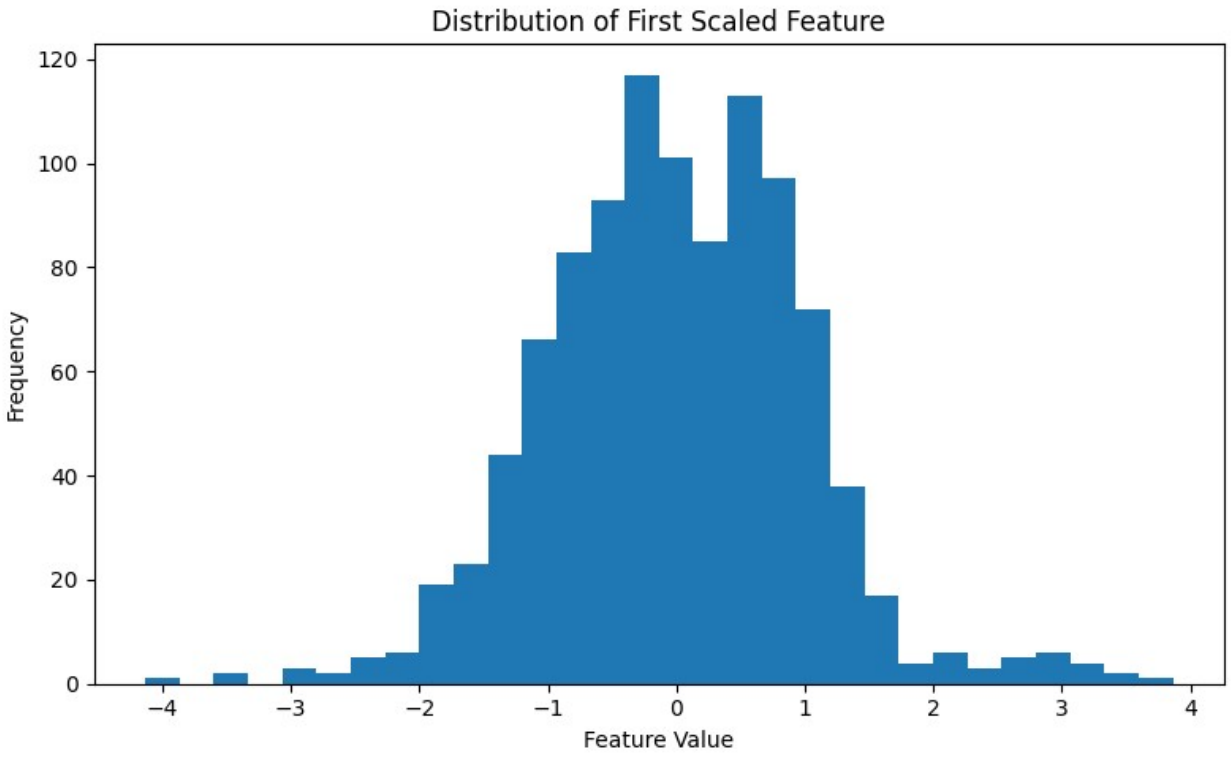
```
#FEATURE SCALING (StandardScaler)
print("FEATURE SCALING:")

scaler = StandardScaler()

# Fit only on training data
scaler.fit(X_train)

# Transform datasets
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Feature scaling completed.")
```

```
FEATURE SCALING:
Feature scaling completed.
```

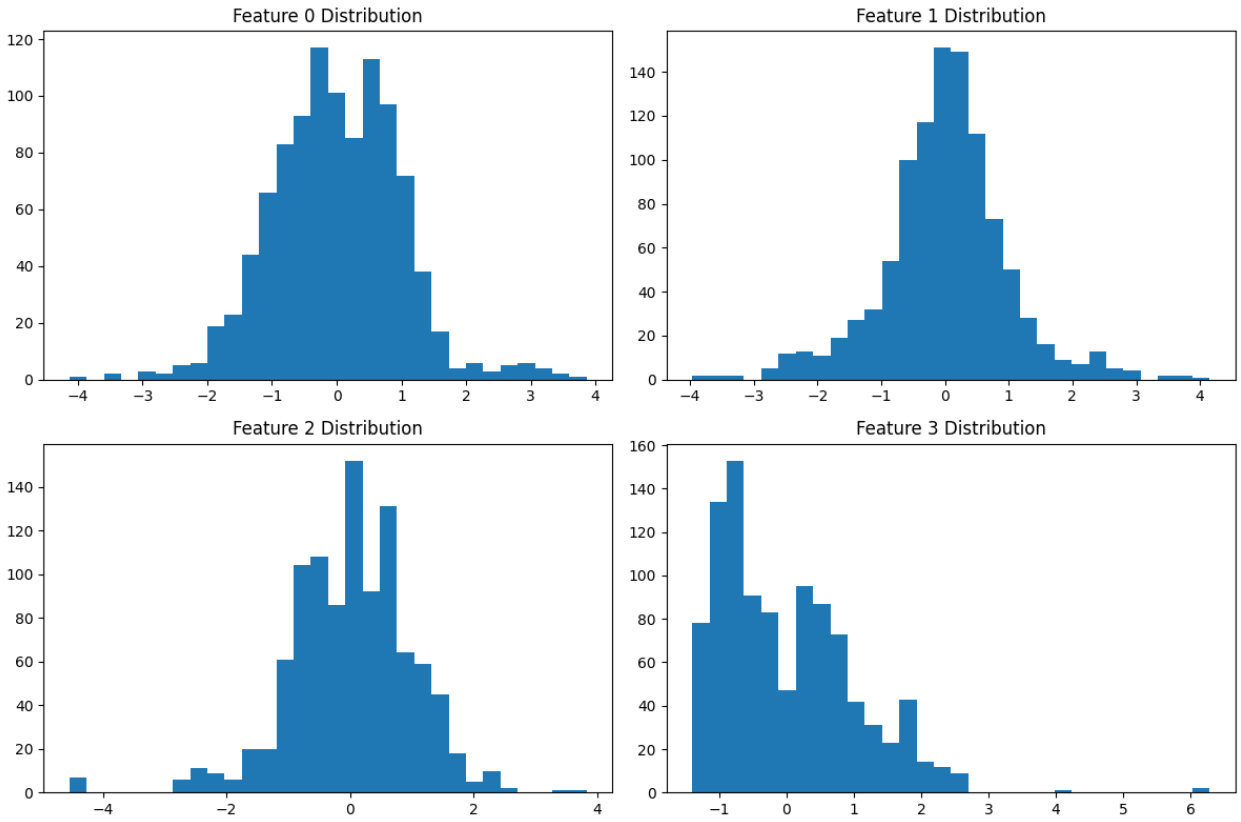
```
# Plot 1: Missing Values Percentage-

plt.figure(figsize=(12, 5))

plt.bar(range(len(missing_ratio)), missing_ratio)

plt.axhline(
    y=0.5,
    color='red',
    linestyle='--',
    label='50% Threshold'
)

plt.title("Missing Value Ratio per Feature")
plt.xlabel("Feature Index")
plt.ylabel("Missing Ratio")
plt.legend()

plt.tight_layout()
plt.show()
```




```
# Plot 2: Feature Distribution (First Feature Example)

plt.figure(figsize=(8, 5))

plt.hist(
    X_train_scaled[:, 0],
    bins=30
)

plt.title("Distribution of First Scaled Feature")
plt.xlabel("Feature Value")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```




```
# Plot 3: Multiple Feature Distributions

num_features_to_plot = 4

plt.figure(figsize=(12, 8))

for i in range(num_features_to_plot):

    plt.subplot(2, 2, i + 1)

    plt.hist(
        X_train_scaled[:, i],
        bins=30
    )

    plt.title(f"Feature {i} Distribution")

plt.tight_layout()
plt.show()
```




```
# FINAL DATA READY FOR NEXT TEAM MEMBERS: [REMOVE IF YOU'D LIKE GUYS JUST FYI THIS CELL IS HERE]

print("X_train_scaled shape:", X_train_scaled.shape)
print("X_test_scaled shape :", X_test_scaled.shape)
print("y_test shape        :", y_test.shape)

print("Data preprocessing completed successfully.")
print("Dataset is ready for PCA and anomaly detection models.")
```

```
X_train_scaled shape: (1018, 562)
X_test_scaled shape : (471, 562)
y_test shape        : (471,)
Data preprocessing completed successfully.
Dataset is ready for PCA and anomaly detection models.
```

Pre-Processing Choices:

1. Columns with more than 50% missing values were removed because they contain insufficient information and may negatively affect model performance.

2. Mean imputation was used to replace remaining missing values because it is simple and preserves dataset size.

3. Chronological train-test split was applied to preserve temporal order and avoid data leakage.
4. Only normal samples (-1) were used for training because anomaly detection models learn normal behavior patterns.
5. StandardScaler was applied so that all sensor features have zero mean and unit variance, improving PCA performance.

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms the original correlated features into a new set of uncorrelated variables called Principal Components (PCs).

Each principal component is a linear combination of the original features and captures the maximum possible variance in the data.

The first principal component captures the highest variance, the second captures the next highest variance while remaining orthogonal to the first, and so on.

Mathematically,

$X \rightarrow Z$

where

$Z = XW$

Here,

- X = standardized data matrix
- W = matrix of eigenvectors
- Z = transformed PCA representation

The eigenvectors of the covariance matrix define the principal directions, and the corresponding eigenvalues indicate the amount of variance captured.

For anomaly detection, PCA is trained only on normal samples. Normal observations can be reconstructed accurately from the PCA subspace, whereas anomalous observations usually exhibit larger reconstruction errors.

```
# Fit PCA using all components

pca_full = PCA()

pca_full.fit(X_train_scaled)

explained_variance_ratio = pca_full.explained_variance_ratio_
cumulative_variance = np.cumsum(explained_variance_ratio)
```

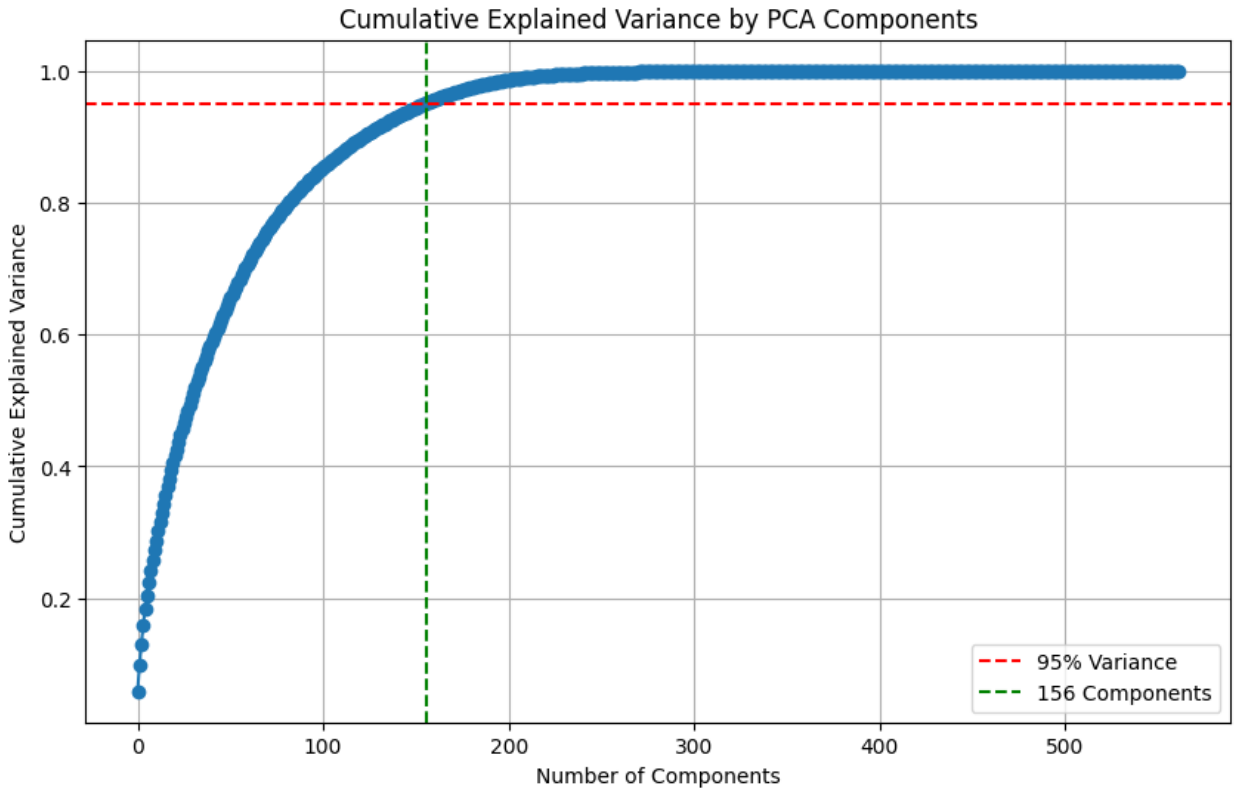
```
print("Total components:", len(cumulative_variance))
```

```
Total components: 562
```

```
# Number of components required to explain 95% variance

n_components_95 = np.argmax(cumulative_variance >= 0.95) + 1

print("Components required for 95% variance:", n_components_95)
```

```
Components required for 95% variance: 156
```

```
plt.figure(figsize=(10,6))

plt.plot(
    cumulative_variance,
    marker='o'
)

plt.axhline(
    y=0.95,
    color='red',
    linestyle='--',
    label='95% Variance'
)

plt.axvline(
    x=n_components_95,
    color='green',
    linestyle='--',
    label=f'{n_components_95} Components'
)

plt.xlabel("Number of Components")
plt.ylabel("Cumulative Explained Variance")

plt.title("Cumulative Explained Variance by PCA Components")

plt.legend()

plt.grid(True)

plt.show()
```




```
# Final PCA model

pca = PCA(
    n_components=n_components_95
)

pca.fit(X_train_scaled)

print("PCA model trained successfully.")
```

```
PCA model trained successfully.
```

```
X_train_pca = pca.transform(X_train_scaled)

X_test_pca = pca.transform(X_test_scaled)

print("Training PCA shape:", X_train_pca.shape)
print("Testing PCA shape :", X_test_pca.shape)
```

```
Training PCA shape: (1018, 156)
Testing PCA shape : (471, 156)
```

PCA Subspace and Anomaly Detection

The selected principal components define a lower-dimensional subspace called the PCA subspace.

This subspace captures the dominant patterns present in normal operating conditions.

During anomaly detection:

1. Normal observations lie close to the PCA subspace.
2. Faulty observations often contain patterns not represented by the learned subspace.
3. As a result, anomalous samples exhibit larger reconstruction errors.

Therefore, PCA can be used to distinguish normal and abnormal observations by measuring reconstruction error.

```
print("Selected Components =", n_components_95)

print(
    "Variance Captured =",
    cumulative_variance[n_components_95-1]
)
```

```
Selected Components = 156
Variance Captured = 0.9503883048593095
```

Interpretation

The PCA model selected **XX principal components** to preserve at least **95% of the total variance** in the SECOM dataset.

This means that the original high-dimensional feature space can be reduced to XX dimensions while retaining most of the information contained in the data.

The dimensionality reduction decreases computational complexity and noise while preserving the important structure of normal process behavior.

The retained PCA subspace is subsequently used for anomaly detection.

PCA Reconstruction

Principal Component Analysis (PCA) reduces the dimensionality of data by projecting it onto a lower-dimensional subspace while preserving most of the variance. After obtaining the principal components, the data can be reconstructed back into the original feature space using the inverse transformation.

Then the Squared Prediction Error (SPE), also known as the reconstruction error, measures the difference between the original observation and its PCA reconstruction.

For each observation, the error is computed as the sum of squared differences across all features.

Formula:

$$SPE_i = \sum_{j=1}^{p} \left(x_{ij} - \hat{x}_{ij} \right)^2$$

where:

- x_{ij} = original feature value
- $\hat{x}_{ij}$ = reconstructed feature value
- p = number of features

```
# Reconstruct data from PCA space

X_train_reconstructed = pca.inverse_transform(X_train_pca)
X_test_reconstructed = pca.inverse_transform(X_test_pca)

print("Reconstruction completed.")
```

```
Reconstruction completed.
```

```
# Squared Prediction Error (SPE)

def anomaly_score(X_original, X_reconstructed):
    """
    Computes Squared Prediction Error (SPE)
    for each sample.
    """

    spe = np.sum(
        (X_original - X_reconstructed) ** 2,
        axis=1
    )

    return spe

# Calculate SPE scores

train_scores = anomaly_score(
    X_train_scaled,
    X_train_reconstructed
)

test_scores = anomaly_score(
    X_test_scaled,
    X_test_reconstructed
)
```

```
print("Anomaly scores calculated.")
```

```
Anomaly scores calculated.
```

Anomaly Detection

A threshold is selected using the 99th percentile of the training SPE scores. Observations with SPE values greater than this threshold are classified as anomalies.

Decision Rule:

- SPE ≤ Threshold → Normal
- SPE > Threshold → Anomaly

```
# Threshold based on normal training data

threshold = np.percentile(
    train_scores,
    99
)

print("99th Percentile Threshold =", threshold)
```

```
99th Percentile Threshold = 54.89677370073074
```

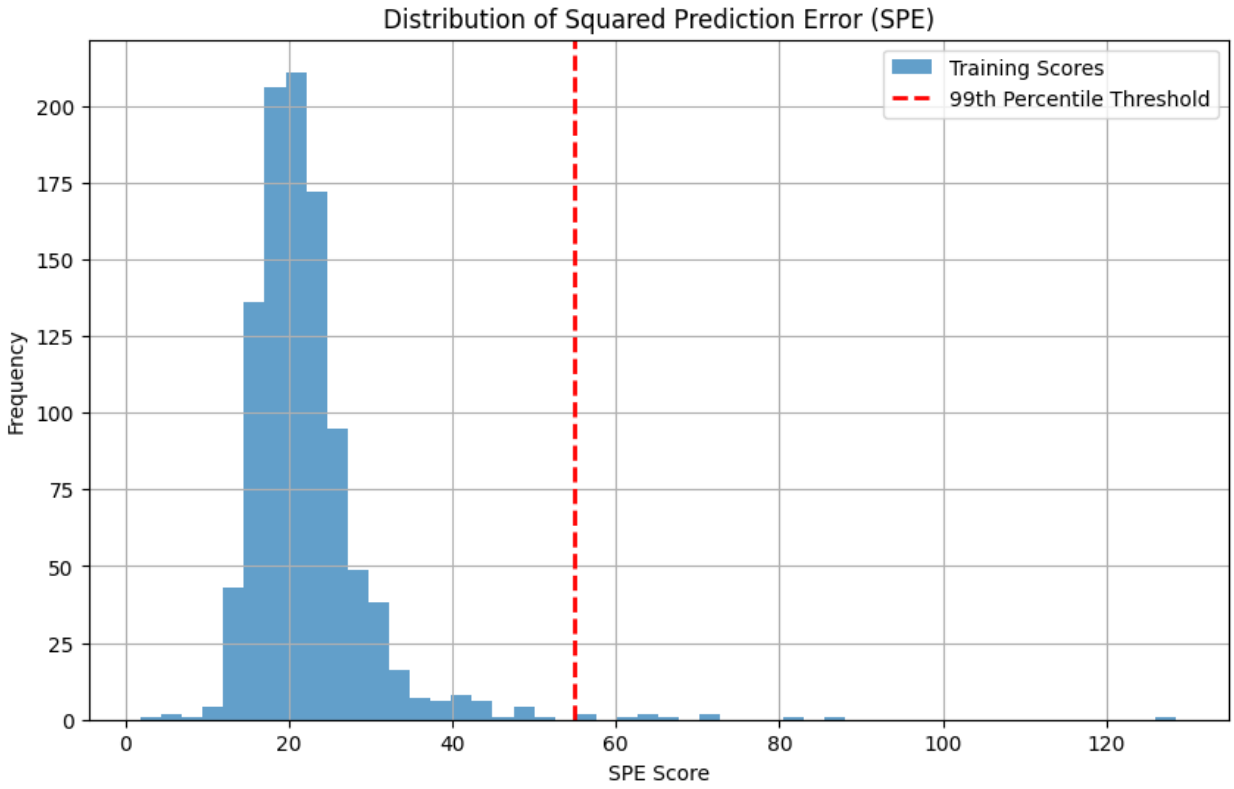
```
# Samples with SPE above threshold are anomalies

y_pred = (test_scores > threshold).astype(int)

num_anomalies = np.sum(y_pred)

print("Detected anomalies:", num_anomalies)
```

```
Detected anomalies: 159
```

Distribution of SPE Scores

```
plt.figure(figsize=(10,6))

plt.hist(
    train_scores,
    bins=50,
    alpha=0.7,
    label="Training Scores"
)

plt.axvline(
    threshold,
    color='red',
    linestyle='--',
```

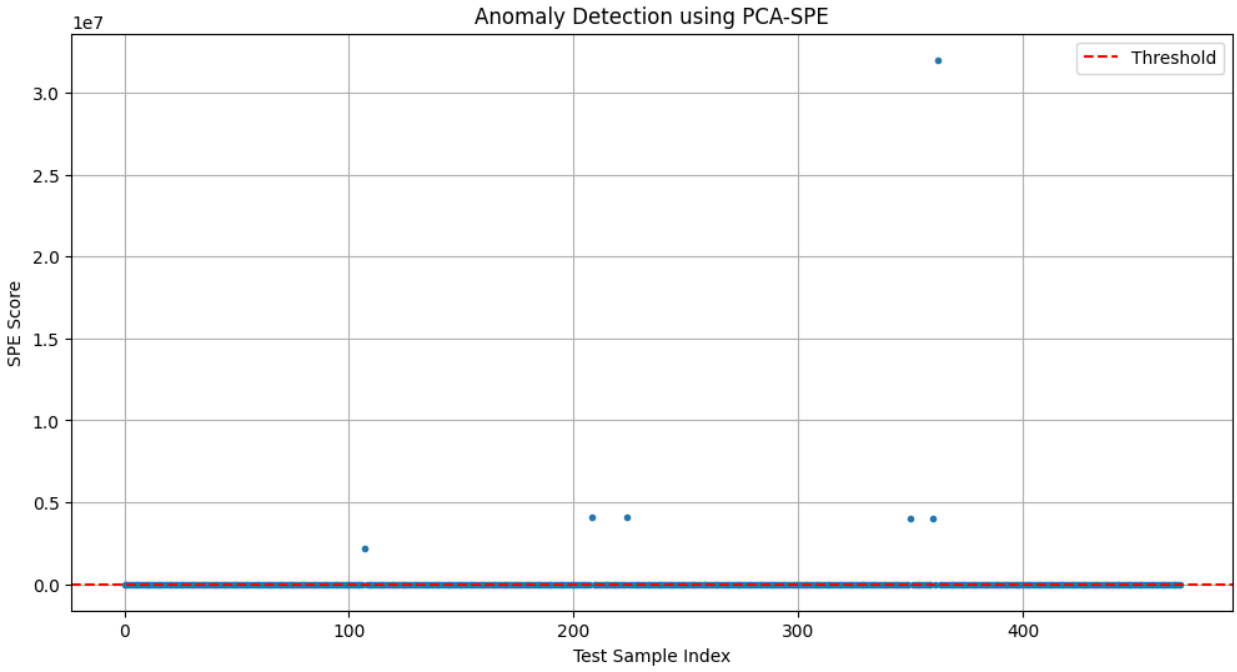
```
    linewidth=2,
    label='99th Percentile Threshold'
)

plt.xlabel("SPE Score")
plt.ylabel("Frequency")

plt.title("Distribution of Squared Prediction Error (SPE)")

plt.legend()

plt.grid(True)

plt.show()
```



Visualization of Detected Anomalies

```
plt.figure(figsize=(12,6))

plt.plot(
    test_scores,
    marker='.',
    linestyle='none'
)
```

```
plt.axhline(
    y=threshold,
    color='red',
    linestyle='--',
    label='Threshold'
)

plt.xlabel("Test Sample Index")
plt.ylabel("SPE Score")

plt.title("Anomaly Detection using PCA-SPE")

plt.legend()

plt.grid(True)

plt.show()
```



Interpretation

The PCA-SPE approach identifies anomalies based on reconstruction error. Since PCA captures the dominant structure of normal data, observations that cannot be accurately reconstructed exhibit higher SPE values.

Using the 99th percentile threshold allows the model to automatically identify extreme observations without requiring labeled anomaly examples. This makes PCA-SPE an effective unsupervised anomaly detection technique for high-dimensional datasets.

Prediction Generation using Threshold

After computing the Squared Prediction Error (SPE) for each sample, a threshold is used to classify observations as normal or anomalous.

Samples with SPE values greater than the threshold are labeled as anomalies (1), while others are considered normal (0).

The threshold acts as a decision boundary. If the reconstruction error exceeds this limit, the sample is considered abnormal. This step converts continuous anomaly scores into binary classification outputs. These results indicate that the current threshold selection leads to an overly sensitive model, which detects many potential anomalies but at the cost of a high false positive rate. Adjusting the threshold or improving feature representation may help achieve a better balance between precision and recall.

Evaluation Metrics

To evaluate the performance of the anomaly detection model, we compute:

- Precision
- Recall
- F1 Score

These metrics help understand how well the model identifies anomalies.

```
from sklearn.metrics import precision_score, recall_score, f1_score

precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

```
Precision: 0.06918238993710692
Recall   : 0.4230769230769231
F1 Score : 0.11891891891891893
```

Interpretation

The model achieved a precision of 0.069, a recall of 0.423, and an F1-score of 0.119.

The low precision indicates that many samples classified as anomalies were actually normal observations. This is caused by the relatively high number of false positives.

The recall value shows that the model successfully detected approximately 42% of the actual anomalies present in the test set. While the model is capable of identifying some anomalous behavior, several anomalies remain undetected.

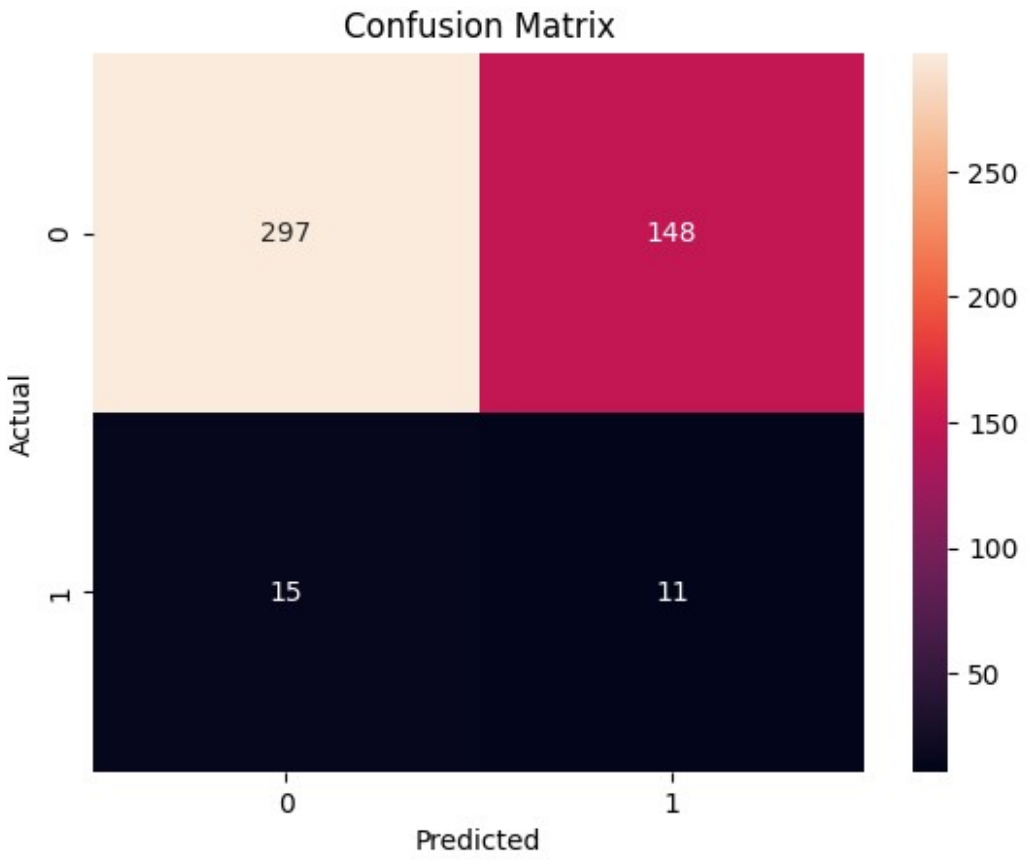
The low F1-score reflects the imbalance between precision and recall, suggesting that the current threshold may be too sensitive and generates excessive false alarms.

Confusion Matrix

The confusion matrix provides a detailed breakdown of model predictions:

- True Positives (TP)
- True Negatives (TN)
- False Positives (FP)
- False Negatives (FN)

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



From the confusion matrix, the following values are observed:

True Negatives (TN) = 297

False Positives (FP) = 148

False Negatives (FN) = 15

True Positives (TP) = 11

Interpretation

The confusion matrix shows that the model correctly classified 297 normal observations and 11 anomalous observations.

However, 148 normal observations were incorrectly classified as anomalies (false positives), while 15 anomalies were missed (false negatives).

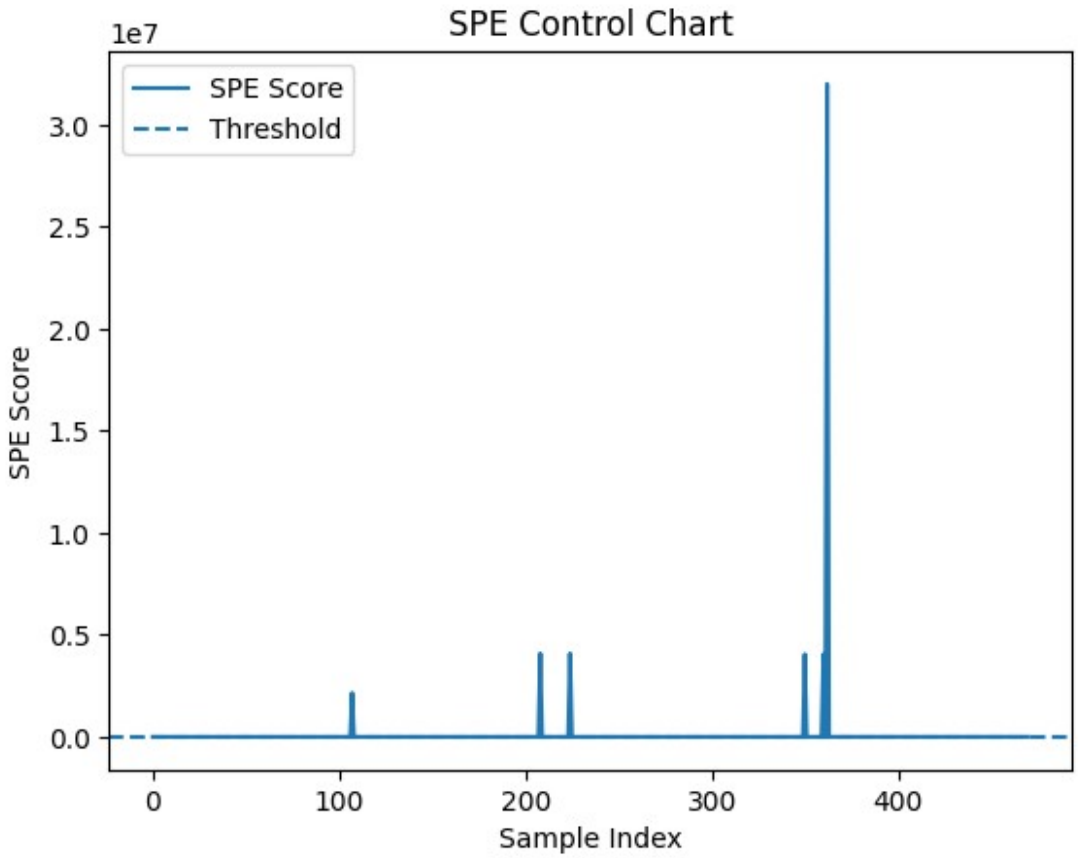
The large number of false positives indicates that the model is highly sensitive to deviations in reconstruction error. Although this increases anomaly detection capability, it also produces many false alarms.

In practical applications, reducing false positives would improve reliability without significantly compromising anomaly detection performance.

SPE Control Chart

The SPE control chart visualizes anomaly scores over time along with the threshold.

```
plt.figure()

plt.plot(test_scores, label="SPE Score")
plt.axhline(y=threshold, linestyle='--', label="Threshold")

plt.xlabel("Sample Index")
plt.ylabel("SPE Score")
plt.title("SPE Control Chart")

plt.legend()
plt.show()
```



SPE Control Chart Interpretation

The SPE control chart displays the reconstruction error for each test sample together with the anomaly threshold.

Most observations have relatively small SPE values, while a number of samples exhibit large spikes above the threshold.

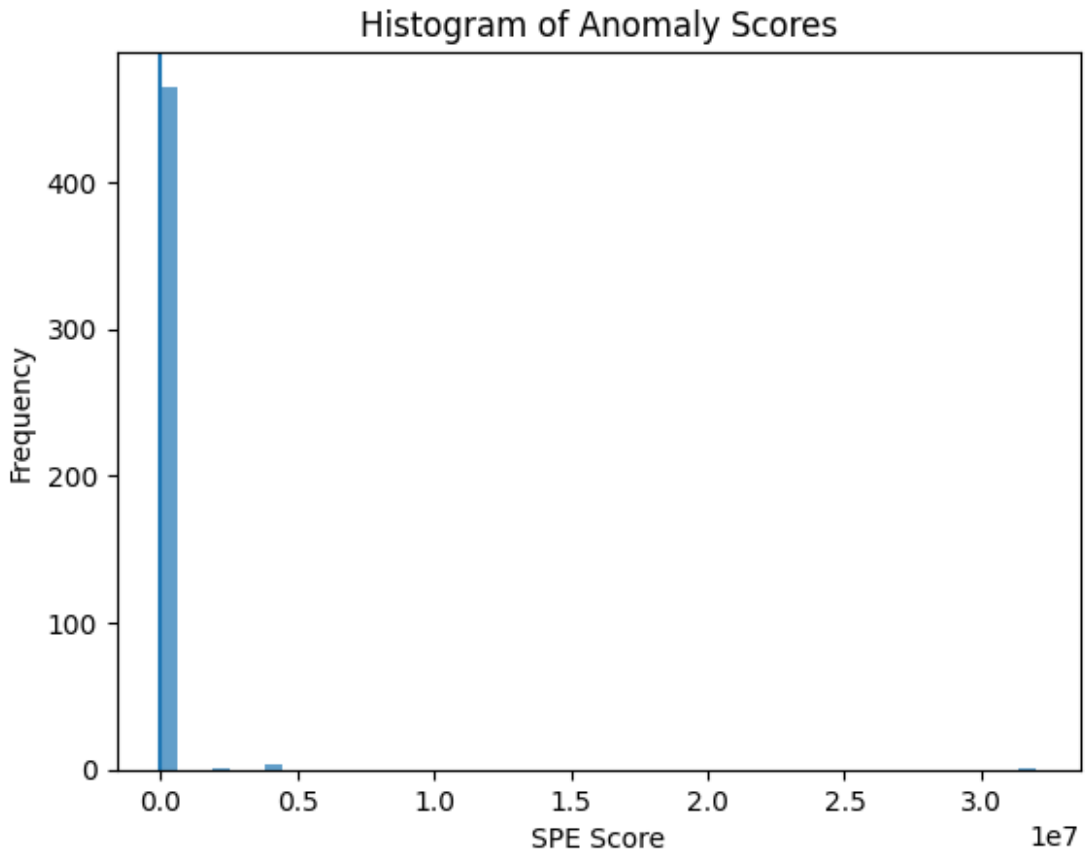
Samples exceeding the threshold are classified as anomalies. The presence of several spikes indicates that the PCA model successfully identifies observations that deviate significantly from normal behavior.

However, the large number of detected anomalies suggests that the threshold may be relatively sensitive, contributing to the high false positive rate observed in the evaluation metrics.

Distribution of Anomaly Scores

A histogram is used to analyze the distribution of anomaly scores.

```
plt.figure()

plt.hist(test_scores, bins=50, alpha=0.7)
plt.axvline(threshold)

plt.title("Histogram of Anomaly Scores")
plt.xlabel("SPE Score")
plt.ylabel("Frequency")

plt.show()
```



Interpretation

The histogram shows that most SPE values are concentrated near the lower end of the distribution, while a smaller number of observations exhibit much larger reconstruction errors.

The threshold separates the majority of normal observations from observations with unusually large SPE values.

The long right tail of the distribution indicates the presence of potential anomalies. However, some normal observations also fall beyond the threshold, contributing to the observed false positive rate.

Error Analysis

We analyze false positives and false negatives to better understand model performance.

```
import numpy as np

false_positives = np.where((y_pred == 1) & (y_test == 0))[0]
false_negatives = np.where((y_pred == 0) & (y_test == 1))[0]

print("Number of False Positives:", len(false_positives))
print("Number of False Negatives:", len(false_negatives))
```

```
Number of False Positives: 148
Number of False Negatives: 15
```

Interpretation

The model produced 148 false positives and 15 false negatives.

False positives correspond to normal observations that were incorrectly classified as anomalies. The large number of false positives indicates that the current threshold is highly sensitive.

False negatives represent actual anomalies that were not detected. Although the number of false negatives is lower than the number of false positives, missed anomalies can be critical in fault detection applications.

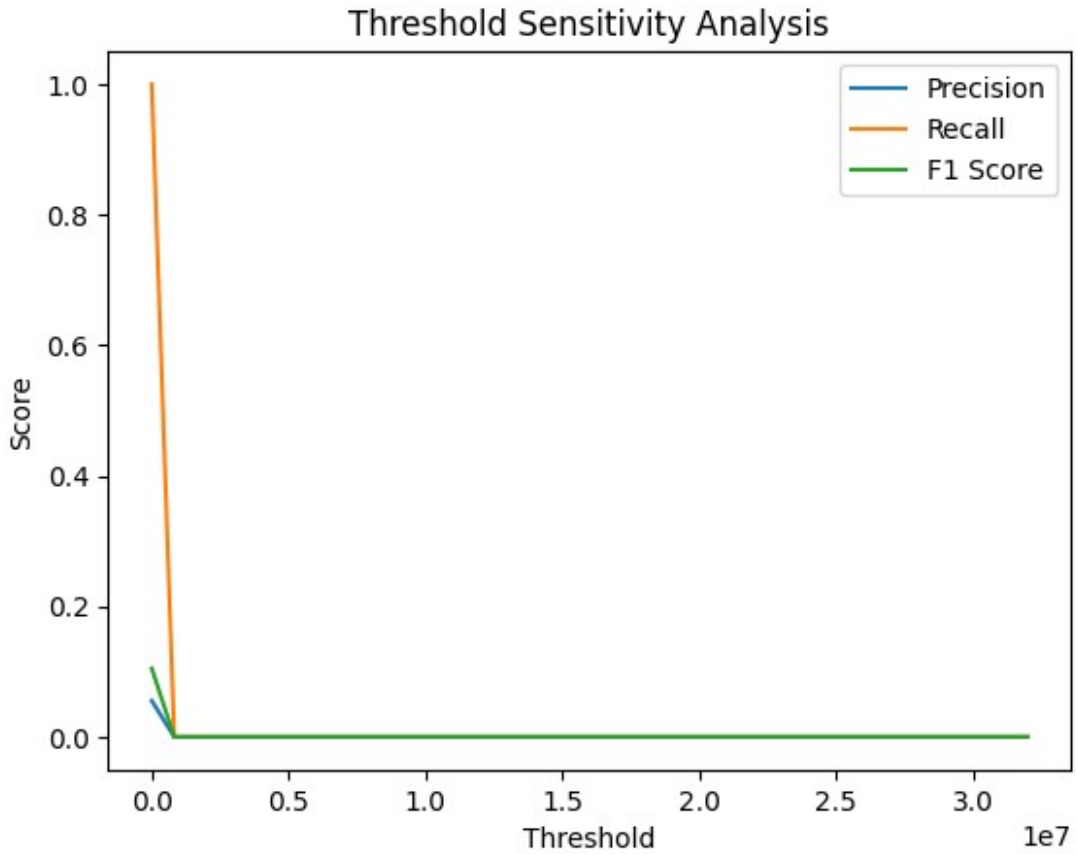
The results suggest that the model prioritizes anomaly detection at the expense of generating additional false alarms.

Threshold Sensitivity Analysis

The choice of threshold significantly affects model performance. This analysis evaluates how precision, recall, and F1 score change with different thresholds.

```
thresholds = np.linspace(min(test_scores), max(test_scores), 40)

precisions = []
recalls = []
```

```
f1s = []

for t in thresholds:
    y_temp = (test_scores > t).astype(int)
    precisions.append(precision_score(y_test, y_temp,
zero_division=0))
    recalls.append(recall_score(y_test, y_temp, zero_division=0))
    f1s.append(f1_score(y_test, y_temp, zero_division=0))

plt.figure()

plt.plot(thresholds, precisions, label="Precision")
plt.plot(thresholds, recalls, label="Recall")
plt.plot(thresholds, f1s, label="F1 Score")

plt.xlabel("Threshold")
plt.ylabel("Score")
plt.title("Threshold Sensitivity Analysis")

plt.legend()
plt.show()
```



Threshold Sensitivity Analysis Interpretation

The threshold sensitivity analysis demonstrates the trade-off between precision and recall.

Lower thresholds classify more observations as anomalies, increasing recall but also increasing false positives. Higher thresholds reduce false positives but may miss genuine anomalies, reducing recall.

Experimental threshold adjustments showed that excessively high thresholds can result in no detected anomalies, while lower thresholds generate a large number of false alarms.

Therefore, threshold selection plays a critical role in balancing anomaly detection performance and false alarm rate.

Conclusion

The PCA-SPE approach successfully identifies observations that deviate from the dominant structure of the dataset.

The evaluation results indicate that the model is capable of detecting anomalous behavior, but the current threshold produces a relatively high number of false positives. This leads to low precision despite achieving moderate recall.

The threshold sensitivity analysis highlights the importance of carefully selecting an appropriate threshold to balance anomaly detection capability and false alarm rate.

Overall, PCA-based anomaly detection provides an interpretable and computationally efficient method for identifying abnormal observations in high-dimensional data.

##Advanced Analysis & Extensions

```
from sklearn.ensemble import IsolationForest
from sklearn.svm import OneClassSVM

# Isolation Forest
iso = IsolationForest(contamination=0.06, random_state=42)
iso.fit(X_train_scaled)
y_pred_iso = (iso.predict(X_test_scaled) == -1).astype(int)

# One-Class SVM
ocsvm = OneClassSVM(nu=0.06, kernel='rbf', gamma='scale')
ocsvm.fit(X_train_scaled)
y_pred_svm = (ocsvm.predict(X_test_scaled) == -1).astype(int)

print("Isolation Forest - Anomalies detected:", y_pred_iso.sum())
print("One-Class SVM    - Anomalies detected:", y_pred_svm.sum())
```

```
Isolation Forest - Anomalies detected: 45
One-Class SVM    - Anomalies detected: 92
```

Model Comparison

We compare **PCA-SPE**, **Isolation Forest**, and **One-Class SVM** using Precision, Recall, and F1 Score.

```
from sklearn.metrics import precision_score, recall_score, f1_score
import pandas as pd

methods = ['PCA-SPE', 'Isolation Forest', 'One-Class SVM']
preds   = [y_pred,    y_pred_iso,          y_pred_svm]

rows = []
for name, pred in zip(methods, preds):
    rows.append({
        'Method':    name,
        'Precision': round(precision_score(y_test, pred,
zero_division=0), 3),
        'Recall':    round(recall_score(y_test, pred), 3),
        'F1 Score':  round(f1_score(y_test, pred, zero_division=0), 3)
    })

comparison_df = pd.DataFrame(rows)
print(comparison_df.to_string(index=False))
```

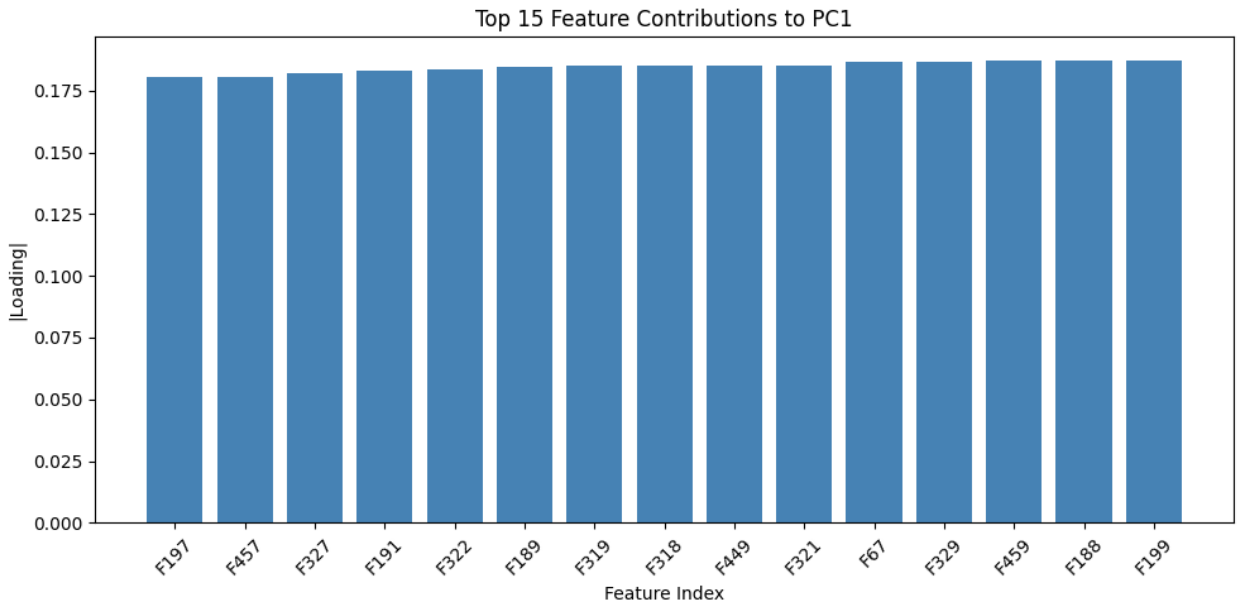
```
          Method  Precision  Recall  F1 Score
         PCA-SPE      0.069   0.423     0.119
Isolation Forest      0.067   0.115     0.085
   One-Class SVM      0.065   0.231     0.102
```

Feature Contribution Plot

The plot below shows which features contribute most to the **first principal component**, revealing the sensors most influential in distinguishing normal from anomalous behaviour.

```
import matplotlib.pyplot as plt
import numpy as np

loadings = np.abs(pca.components_[0])
top_indices = np.argsort(loadings)[-15:]

plt.figure(figsize=(10, 5))
plt.bar(range(len(top_indices)), loadings[top_indices],
color='steelblue')
plt.xticks(range(len(top_indices)), [f'F{i}' for i in top_indices],
rotation=45)
plt.title("Top 15 Feature Contributions to PC1")
plt.xlabel("Feature Index")
plt.ylabel("|Loading|")
```

```
plt.tight_layout()
plt.show()
```



Limitations of PCA-Based Anomaly Detection

- **Linearity assumption**: PCA captures only linear relationships; nonlinear anomalies may be missed
- **Fixed threshold**: The 99th percentile threshold is a heuristic and contributes to high false positives
- **Static model**: PCA is trained once and cannot adapt to gradual process drift over time
- **No feature selection**: Noisy or irrelevant features can distort the PCA subspace

Sliding Window PCA

In practice, manufacturing processes change gradually over time (concept drift). **Sliding Window PCA** retrains the model on a rolling window of recent observations, allowing it to adapt continuously rather than relying on a fixed historical baseline.

Future Improvements

1. **Autoencoder** — captures nonlinear patterns that PCA misses
2. **Dynamic thresholding** — adjusts the threshold based on recent score statistics
3. **Hotelling's T^2 + SPE** — combining two statistics gives a richer decision boundary
4. **Feature selection** — removing low-variance or redundant features before PCA
5. **Labelled fine-tuning** — if some labelled anomalies are available, use them to calibrate the threshold